

Objectifs visés :

- ✓ Configuration matérielle et logicielle d'une structure de commande de type API
- ✓ Service IO-Scanning sous Ethernet/IP
- ✓ Programmation d'un automate intégrant un combinatoire cyclique et des spécifications séquentielles à l'aide d'une structure de machine de MEALY

1. Système commandé (PO, PC)

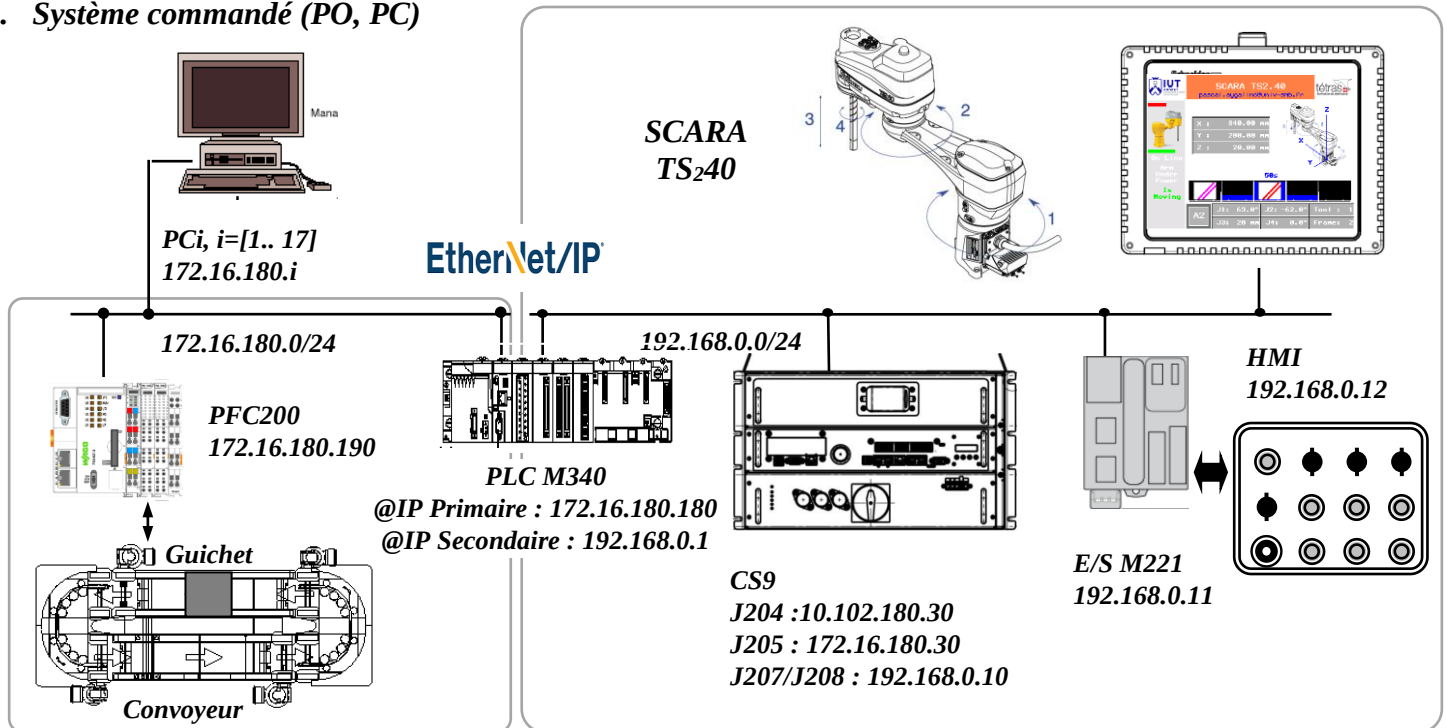


Figure 1. Cellule robotisée de la salle C180 : (PLC M340, CS9, M221, HMI et PFC200)

La cellule sur laquelle s'effectueront les tps (cf. figure 1) est composée au niveau matériel :

- ✓ d'un robot **Staubli Scara TS240 collaboratif** (4 axes) et de son contrôleur **CS9**, équipé de son **pendant MCP** et de son **sélecteur de modes de marche WMS9**,
- ✓ d'un convoyeur **TS2plus Bosch RexRoth** sur lequel circule des **palettes** portant des produits,
- ✓ d'un **contrôleur PFC200** du constructeur **WAGO** de type **Little Endian** qui gère une partie du convoyeur mais qui se comporte comme un **îlot d'E/S** accessible via **Ethernet/IP** pour la commande du **Guichet**,
- ✓ d'un **contrôleur Schneider** de type **M221** dont la programmation réalisée permet l'accès au **pupitre opérateur** via **Ethernet/IP** et de fournir les informations de supervision à l'écran **HMI** à l'aide du protocole **Modbus-Tcp**,
- ✓ d'un **automate Schneider** de type **M340** dont l'orientation des données est de type **Little Endian**. Il est équipé de **deux coupleurs réseaux BMX NOC 0401.2** qui permettent d'obtenir :
 - pour le premier : une communication de type **Ethernet** avec les stations de développement (**PCs**) sur le **réseau privatif de la salle C180 (classe B : 172.16.180.0/24)**. Elle est nécessaire pour le **téléchargement du code** dans l'automate et la **surveillance de son exécution** en mode connecté. Il permet aussi de scruter le **contrôleur PFC200** afin d'établir via **Ethernet/IP** la commande du **Guichet** du convoyeur sur lequel le robot **Scara** viendra saisir les pièces,
 - pour le second : un réseau de type **Ethernet/IP** pour **les échanges périodiques** à réaliser entre l'**API M340, la baie CS9 et le contrôleur M221**.

Cette cellule a été réalisée au **département Geii** de l'**IUT d'Annecy** : **Michèle Alligier** pour la partie câblage électrique, **Pascal Aygalinc** pour sa conception, sa réalisation mécanique et sa programmation.

1.1 Partie commande : API C180-M340

La station **API** est un **automate programmable de conception modulaire**. Elle est basée sur un processeur de type **M340** du constructeur **Schneider** (<http://www.e-catalogue.schneider-electric.fr>) et est constituée (cf. figure 3) :

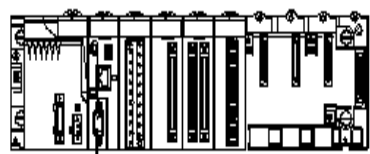


Figure 3 : station API

- i) d'un rack extensible : **BMX XBP 0600**
- ii) d'une alimentation **DC 24V isolée BMX CPS 2010**
- iii) d'un processeur **BMX P34 20102 (firmware v3.3)**
- iv) de **deux coupleurs 4 ports Ethernet BMX NOC 0401.2** offrant les deux protocoles : **Ethernet/IP** et **Modbus-Tcp**

1.2 Le Poste Guichet

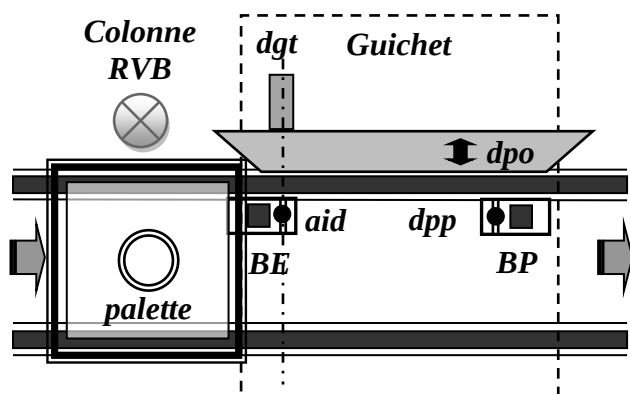


Figure 2 : Le guichet

Ce poste **guichet** est composé :

- ✓ de **deux bloqueurs (actif au repos)** qui permettent de maintenir **la palette** dans le poste (**BP**) et de l'isoler du flux de produit (**BE**) venant du convoyeur (supposé toujours en marche),
- ✓ d'une colonne lumineuse **RVB** afin de signaler son état (libre, occupé, en phase d'occupation, de libération)

Son **fonctionnement** dans le mode **F1** est identique à celui d'un **sas** puisque ce poste est de **capacité unitaire**. Pour qu'une palette puisse entrer dans le guichet, celui-ci doit être libre.

L'instrumentation ainsi que la nature des actionneurs font l'objet des tableaux suivants :

Actionneur	Référence Utilisateur	Nature de l'actionneur
BE	nqxBE	Abaissier le Bloqueur d'Entrée (distributeur monostable)
BP	nqxBP	Abaissier le Bloqueur du Poste (distributeur monostable)
CR	nqxCR	Colonne lumineuse RVB : Composante Rouge
CV	nqxCV	Colonne lumineuse RVB : Composante Verte
CB	nqxCB	Colonne lumineuse RVB : Composante Bleue

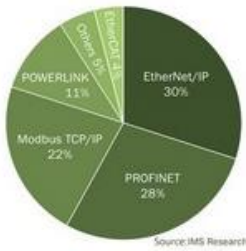
Capteur	Référence Utilisateur	Information portée
aid	nixAid	capteur validant l'information du capteur de détection de gobelet
dgt	nixDgt	capteur de détection de gobelet (information valide si aid est actif)
dpp	nixDpp	capteur de détection d'une palette en poste
dpo	nixDpo	capteur de détection que le poste est occupé

- Nota :**
1. $(aid, dgt) = (1, 0) \Leftrightarrow$ la palette est vide, $(aid, dgt) = (1, 1) \Leftrightarrow$ la palette porte un gobelet.
 2. Les **palettes** sont **mécaniquement prévues** pour que le **blocage d'une palette en entrée** du guichet soit commandé lors de la **détection de la palette précédente**.
 3. Les variables seront obtenues par un service d'**IO-Scanning** : **préfixe n**. Elles sont de type **BOOL**. Elles sont **non forcables** et leur **dynamique (front)** est **non accessible**.

Ce premier tp de prise en main portera sur la configuration du réseau **Ethernet/IP** et sur la communication à réaliser entre l'**API M340** et le contrôleur **PFC200**. Ceci permettra d'effectuer une première ébauche du mode de production normale (**mode F1**) car on s'intéressera uniquement au poste **Guichet** situé sur le convoyeur et **en isolation**.

2. Configurations logicielles

2.1 EtherNet/IP



EtherNet/IP (*EtherNet Industrial Protocol*) est une **norme de bus de terrain ouverte**, basée sur **CIP** (*Common Industrial Protocol*), développée par **Rockwell Automation** et par l'**ODVA** (*Open DeviceNet Vendor Association*). Elle est utilisée principalement dans le domaine de l'automatisation et fait partie des **réseaux Ethernet déterministes**. Etant **bien plus rapide** que **Modbus-Tcp**, **EtherNet/IP** tend à le remplacer car il n'impose pas de contrainte particulière pour son intégration.

En effet, **EtherNet/IP** prend en charge *Ethernet*, *Internet Protocol* et les protocoles de transport **TCP** (*Transport Control Protocol*) et **UDP** (*User Datagram Protocol*) du **réseau TCP/IP**. Cette **compatibilité** avec les couches du **réseau TCP/IP** permet à **EtherNet/IP** une intégration facile dans les réseaux classiques et dans l'installation à commander (*cette norme ne requiert pas d'actifs réseaux spécifiques (switch, routeur, ...)*). **EtherNet/IP** offre des vitesses standard (10, 100Mbps et 1Gbps) sur différents supports (*cuivre, fibre*), une topologie en bus, en étoile ou en anneau. Dans ce dernier cas, il est possible d'implémenter le protocole « **DLR** » (*Device Level Ring*) afin d'empêcher une panne de réseau totale en raison d'une rupture de câble ou la panne d'un participant. Quelques éléments de terminologie d'**EtherNet/IP** vous sont donnés ci-dessous :

- ✓ **Scanner / Originator** : désigne le « **maitre** » qui a l'initiative des échanges (**API** pour ce tp),
- ✓ **Adaptater / Target** : désigne « **l'esclave** » qui répond aux sollicitations du maitre (ilot I/O)
- ✓ **RPI** : périodicité des échanges I/O (**Requested Packet Interval**)
- ✓ **O -> T** (*Originator -> Target*) : **OUT**put (du « **maitre** » vers « **l'esclave** » : **Consommation**)
- ✓ **T -> O** (*Target -> Originator*) : **IN**put (de « **l'esclave** » vers le « **maitre** » : **Production**)
- ✓ **ForwardOpen** : service acyclique pour demander une connexion pour un échange de données

Pour bénéficier du support **Ethernet/IP** sur les **API BMX P34** (série **M340**) du constructeur **Schneider**, le rack doit être équipé d'un coupleur spécifique : **BMX NOC401.2**. Ce coupleur utilise l'espace mémoire conventionnel de l'**API (%MWi)** pour stocker :

- ✓ dans des mots mémoire en **entrée (IN)** : l'état de **santé** de ses connexions vers les équipements « **Target** » (**HEALTH_BITS_IN** : ARRAY[0..31] OF BYTE), suivi de la **zone image des données de Production (INput)** dont la **taille** est définie par l'utilisateur (**XX+1**) en fonction **des équipements** qu'il désire connecter.
- ✓ dans des mots mémoire en **sortie (OUT)**: la **mise en service** de ses connexions (**CONTROL_BITS_OUT** : ARRAY[0..31] OF BYTE (actif à 0), suivie de la **zone image des données de Consommation (OUTput)** dont la **taille** est aussi définie par l'utilisateur (**YY+1**) pour les mêmes raisons.

NOC_ETHIP_IN
(%MW_r)

NOC_ETHIP_OUT
(%MW_w)

WORD (16 bits)

HEALTH_BITS_IN
(ARRAY[0..31] OF BYTE)

Zone image des données de
Production (INput)
T->O
(ARRAY[0..XX] OF WORD)

CONTROL_BITS_OUT
(ARRAY[0..31] OF BYTE)

Zone image des données de
Consommation (OUTput)
O->T
(ARRAY[0..YY] OF WORD)

Nota : Ces **deux zones (IN, OUT)** ne doivent pas se chevaucher. Elles peuvent être **séparées** ou **contiguës**.

Sous **Ethernet/IP**, la **configuration des services** accessibles d'un **équipement** s'effectue à l'aide d'un fichier texte d'extension **eds** (**EDS** : *Electronic Data Sheet*). Suivant qu'il est **générique** ou/et **paramétrable**, on a la possibilité de l'adapter aux besoins de l'application et/ou de l'équipement (**RPI**, **nom** et **nombre des données**

produites (**T->O : IN**) exprimées en octet, nom et nombre des données consommées (**O->T : OUT**) exprimées aussi en octet, ...).

Nota : Il existe un outil pour concevoir ses propres fichiers **EDS**. Il est disponible à partir du lien suivant : <https://www.odva.org/software/EZ-EDS-Download>

2.1.1 A partir des informations fournies sur l'organisation mémoire des coupleurs **BMX NOC 0401.2**, établir la cartographie de l'espace mémoire **%MW** de l'**API** si on effectue la configuration suivante :

- ✓ Pour 1^{er} coupleur **BMX NOC 0401.2 : NOC_ETHIP_CONVOYEUR**, on décide de réserver **32 mots** en entrée à partir de l'adresse **%MW0** et **32 mots** en sortie juste après. Sur ce réseau, un seul équipement sera scruté (**PFC200**). Il produit (**Input T->O**) **12 octets** et en consomme **4 (Output O->T)**.
- ✓ Pour 2^{ème} coupleur **BMX NOC 0401.2 : NOC_ETHIP_ROBOT**, son image mémoire sera contiguë à celle du premier coupleur et devra comprendre **128 mots** en entrée et **128 mots** en sortie. Les données produites et consommées par les 2 équipements (**baie CS9** et **contrôleur M221**) vous seront données lors des prochains TP.

2.1.2 L'empreinte mémoire des données produites et consommées par le contrôleur **PF200** est donnée par les tableaux suivants (les cases grisées sont réservées pour une utilisation future)

T->O	rang du bit	Guichet_IN
Octet 0	0	aid
	1	dgt
	2	dpp
	3	dpo
Octet 1		
Octet 2		
...		
Octet 11		

O->T	rang du bit	Guichet_OUT
Octet 0	0	BE
	1	BP
	2	CR
	3	CV
	4	CB
Octet 1		
...		
Octet 3		

En utilisant **les techniques de chevauchement et d'alias**, donner les adresses mémoire à utiliser pour obtenir ces informations en utilisant la cartographie établie précédemment. On rappelle que **%MWi.j** représente l'adresse du bit de rang **j** du mot mémoire d'adresse **%MWi**.

2.2 Chien de garde de l'API et Période de scrutation

Sachant que la vitesse du convoyeur est de l'ordre de 0.2 m/s, qu'un capteur magnétique assure la détection d'une palette par-dessous sur une zone de 1 cm et que la bascule signalant un guichet occupé est de 28 cm :

2.2.1 Tracer les chronogrammes représentant la durée d'activation des capteur **aid** et **dpo** signalant la détection/présence d'une palette dans le guichet.

2.2.2 En déduire à partir de ces deux chronogrammes, le capteur qui fixe la dynamique du procédé, la configuration du chien de garde de l'API et la période de scrutation.

3. Mode F1 : Mode Production Normale

3.1 Etablir la commande des bloqueurs du guichet afin d'obtenir une capacité unitaire de ce dernier sans faire de distinction entre les palettes pleines ou vides. La colonne lumineuse RVB sera éteinte.

3.2 Compléter cette commande afin de maintenir en poste pendant **T#3s** uniquement les palettes portant un gobelet. La capacité unitaire doit toujours être vérifiée.

3.3 En déduire le traitement utilisateur à réaliser pour ces deux cahiers des charges (cf. **Annexe**).

4. Partie TP

Le mise en œuvre associée à ce **TD/TP** reste générale et ne se substitue pas aux **instructions de sécurité** détaillées contenues dans les **manuels** des produits **Schneider Electric SA** et leurs caractéristiques techniques. Ces documents doivent **être lus attentivement et appliqués** avant de se lancer dans la partie **TP**. On rappelle que le **non-respect** de ces **instructions** peut provoquer **la mort, des blessures graves** ou **des dommages matériels**.

4.1 Travail préparatoire : Importation de fichiers **eds** décrivant des équipements

Comme l'importation de **fichiers EDS** à **Control Expert** réclame des **droits d'administrateur sur les PCs**, ce travail a été réalisé après l'installation de ce logiciel. Si votre activité professionnelle vous invite à le faire, il s'effectue sous **Control Expert** de la façon suivante :

- i) tout d'abord ouvrir le navigateur **DTM** (Menu : **Outils->Navigateur de DTM**)
- ii) sélectionner un réseau,
- iii) puis via un clic droit dans, menu **Equipement -> Fonctions supplémentaires** choisir l'option **Ajouter un fichier EDS à la bibliothèque**,
- iv) sélectionner le fichier **EDS** à ajouter (bouton ...), puis effectuer son import (bouton **OK**).
- v) enfin, mettre à jour le catalogue des **DTMs** (Menu : **Outil->Catalogue matériel**, onglet **Catalogue DTM**, bouton **Mise à jour**)

4.2 Après s'être connecté(e) à une station PC, créer sous votre dossier personnel **P:** un répertoire de travail pour ce module. Sous ce répertoire, copier votre squelette personnalisé disponible sous « **U:/Documents/LicencePro/RIF/UE2/AutomatismePourLaRobotique/Squelette** » qui établit **uniquement** :

- ✓ Le choix du **processeur** par rapport à celui de la station **C180-M340 : BMX P34 20102** (firmware 3.30)
- ✓ Le choix du **rack** : un **rack non extensible 6 positions BMX XBP 0600**,
- ✓ Le **nom** du projet : il est spécifique à chacun d'entre vous car il fait référence à votre nom.

4.3 Via un double clique sur votre squelette, ouvrir votre projet à l'aide de **Control Expert**. Si une **boîte de dialogue** vous invite à saisir un mot de passe. Cliquer sur le bouton **Annuler** afin d'éviter cette protection.

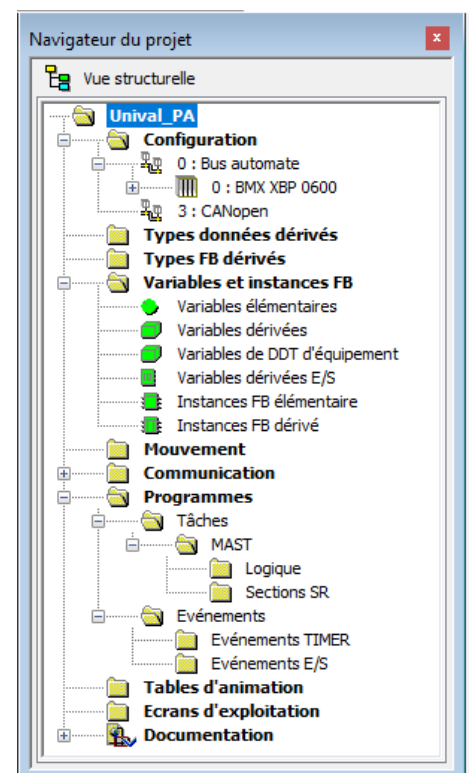
4.4 Ouvrir les différents dossiers du **Navigateur du Projet** comme le montre la figure 4.

C'est en double-cliquant sur chacune des différentes rubriques du **Navigateur** que vous pourrez les renseigner. On respectera l'ordre suivant pour établir l'application :

- 1) Nom de l'application (par défaut **Projet**)
- 2) Configuration matérielle/logicielle
- 3) Création et configuration des tâches
- 4) Déclaration de type Utilisateur
- 5) Edition des objets manipulés par les traitements.
- 6) Création des sections (ou POU's) associées aux différentes tâches
- 7) Ecrans d'exploitation
- 8) Dossier et Documentation

Nota : On prendra pour habitude de sauver l'application régulièrement, au minimum après chaque rubrique.

Figure 4 :



4.5 Effectuer la configuration matérielle du **rack** à l'aide de l'éditeur « graphique » de configuration (double clique sur le dossier **Configuration** de l'explorateur du projet).

4.5.1 Renseigner les différents emplacements (slots) par les équipements qu'ils contiennent, à savoir :

Emplacement n°1	BMX NOC 0401.2	module 4 ports Ethernet RJ45 10/100
Emplacement n°2	BMX NOC 0401.2	module 4 ports Ethernet RJ45 10/100

L'ajout d'un équipement **BMX NOC 0401.2** fait apparaître une boîte de dialogue à plusieurs onglets :

Pour l'emplacement **n°1**, saisir :

- ✓ **Onglet Général** : pour la gestion des noms de **DTMs** : **NOC_ETHIP_CONVOYEUR**
- ✓ **Onglet Information sur le protocole** : sélectionner le protocole **CIP (EtherNet/IP)**. C'est normalement celui-là qui est sélectionné par défaut.

Pour l'emplacement **n°2**, saisir :

- ✓ **Onglet Général** : pour la gestion des noms de **DTMs** : **NOC_ETHIP_ROBOT**
- ✓ **Onglet Information sur le protocole** : sélectionner le protocole **CIP (EtherNet/IP)**.

4.5.2 Configurer les équipements :

- ✓ Configuration du **processeur** : configurer la voie **CANOpen** en ne réservant aucun objet **%M** et **%MW** pour ce réseau en entrée et en sortie (double clique sur la prise **DB9** du réseau **CanOpen** du processeur pour faire apparaître la boîte de dialogue de configuration de ce réseau).
- ✓ Configuration des coupleurs **BMX NOC 0401.2** : **Se référer à votre cartographie pour fournir les différentes adresses à spécifier lorsque l'on effectue un double clique sur eux.**

4.5.3 Sauver votre application. La compiler et vérifier avec l'éditeur de données (variables dérivées) que les zones mémoires allouées aux coupleurs **BMX NOC 0401.2** répondent à vos attentes. On notera que 4 types de données ont été créés pour représenter les 2 coupleurs.

4.6 Configuration des réseaux **NOC_ETHIP_CONVOYEUR** et **NOC_ETHIP_ROBOT**

4.6.1 Configuration des **adresses IP** des interfaces **BMX NOC 0401.2**

- ✓ Ouvrir le **navigateur de DTM** : Menu : **Outils->Navigateur de DTM**. Double cliquer sur le réseau pour obtenir sa fenêtre de configuration.
- réseau **NOC_ETHIP_CONVOYEUR** :
 - Configurer **les propriétés de la voie** par rapport au plan d'adressage de la **figure 1** :
rubrique TCP/IP : @IP : 172.16.180.180, masque : 255.255.255.0, passerelle : 172.16.180.254.
- réseau **NOC_ETHIP_ROBOT** : La configuration par défaut répond à nos attentes. Vérifier néanmoins.
 - **rubrique TCP/IP : @IP : 192.168.0.1, masque : 255.255.255.0, pas de passerelle (0.0.0.0).**

4.6.2 **Ajout et configuration des équipements**

- ✓ réseau **NOC_ETHIP_CONVOYEUR** :
 - Via un clique droit sur le réseau **NOC_ETHIP_CONVOYEUR**, sélectionner l'option **Ajouter**. Dans la liste, l'équipement correspondant au **PFC200** a pour nom : **WAGO 2759-101 EN/IP Revision 1.1 (from EDS)**. Valider en cliquant sur le bouton « **Ajouter DTM** ». Pour la gestion des noms de **DTMs**, prendre : **guichet**.
 - Sur le **navigateur de DTM** :
Sélectionner l'équipement **guichet** et ouvrir sa configuration via un double clique,
Adapter le service « **Exclusive Owner** » à notre application :

- Pour le paramètre **RPI**, prendre une période compatible avec la périodicité des cycles établie en 2.2.
- Pour la rubrique **Entrée T->O**, modifier la **Taille des entrées** à **12 octets**
- Pour la rubrique **Sortie O->T**, modifier la **Taille des sorties** à **4 octets**

Appuyer sur le bouton « **Appliquer** » pour valider vos saisies et fermer la fenêtre.

Dans la liste des équipements sur le réseau **NOC_ETHIP_CONVOYEUR** figure en position **[003] guichet**. Le sélectionner puis :

- **Onglet Paramétrage de l'adresse** : modifier l'adresse **IP** donnée par défaut (172.16.180.181) afin qu'elle corresponde à celle du **PFC200** du plan d'adressage de la figure 1 : **172.16.180.190**
- **Onglet Propriétés** : choisir (sur le bas de la fenêtre) au niveau de la **Gestion des items**, le mode **d'importation manuel**. Il permet de définir des symboles utilisateur pour mapper les données produites et consommées. Valider ce choix avec le bouton « **Appliquer** ».

En cliquant sur **Items**, il vous est possible maintenant de donner des noms aux différents bits produits (onglet **Entrée (bit)**) et consommés (onglet **Sortie (bit)**). Comme ces noms correspondent à des noms de champs des enregistrements **T_guichet_IN** et **T_guichet_OUT** créés par le logiciel, on utilisera comme préfixe **m_nix** pour les entrées, **m_nqx** pour les sorties. Se reporter au paragraphe 1.2 donnant l'empreinte mémoire des données produites et consommées par le contrôleur **PF200** pour en effectuer la saisie. La valider avec le bouton « **Appliquer** ».

4.6.3 *Sauver votre application. La compiler et vérifier avec l'éditeur de données (variables dérivées) que les zones mémoires allouées pour l'équipement ajouté répondent à vos attentes, à savoir :*

- ✓ pour les entrées (**T->O**) :
guichet_IN de type **T_guichet_IN** d'adresse de base **%MW16**.
- ✓ pour les sorties (**O->T**):
guichet_OUT de type **T_guichet_OUT** d'adresse de base **%MW48**.

4.6.4 *Appeler l'enseignant afin qu'il vérifie votre configuration avant tout téléchargement dans l'API*

4.7 *Pour cette application, une seule tâche (**Mast**) suffit. Editer les propriétés de cette tâche (dossier **MAST** menu contextuel **Caractéristiques**) afin de spécifier le **chien de garde** et son **mode de fonctionnement**. On prendra pour valeurs celles établies en 2.2.*

4.8 *Créer deux sections de programme sous la rubrique **Logique : F1_ProductionNormale** en langage **SFC**, **F1_ProductionNormalePost** en langage **ST**. Editer ces sections conformément à votre analyse faite en 3.3.*

4.9 *Sous **Control Expert**, chaque section de programme peut être analysée afin de détecter les erreurs syntaxiques. Cette analyse (menu **Génération -> Analyser**) correspond à une **compilation**. **L'édition de liens** (menu **Génération -> Générer**) permet d'obtenir la totalité du code associé au projet. A ce stade, pour effectuer la recette de l'application (test de sa conformité par rapport au cahier des charges), on peut :*

- ✓ Soit télécharger le code sur un **API virtuel**, et simuler le fonctionnement de la partie opérative pour vérifier le bon fonctionnement de l'automatisme. La génération du code doit se faire avec l'option **Mode Simulation** du menu **Automate**.
- ✓ Soit télécharger le code sur l'**API relié au procédé physique**. La génération du code doit se faire avec l'option **Mode Standard** du menu **Automate**.

*Après avoir choisi l'option **Mode Standard** du menu **Automate**, générer le code de votre projet. Définir l'adresse de l'automate et le protocole à utiliser pour effectuer le téléchargement (**172.16.180.180**,*

protocole **TCP-IP**). Après s'être connecté à l'**API**, télécharger votre projet dans ce dernier et procéder à la recette de ce mode. Pour cela, mettre l'**API** en mode **RUN**.

- 4.10 Faire un bilan mémoire de l'application (menu **Automate**, option **Utilisation de la mémoire**). Interpréter le rôle des différentes zones. Conclure sur la concision du code généré.
- 4.11 Modifier votre programme utilisateur afin d'obtenir sur la colonne lumineuse **RVB** les animations suivantes :

Etat du Guichet	Composante Rouge	Composante Verte	Composante Bleue
Vide	Eteinte	Allumée en permanence	Eteinte
En cours d'occupation	Clignotante 1 seconde	Clignotante 1 seconde	Eteinte
Plein	Allumée en permanence	Eteinte	Eteinte
En cours de libération	Eteinte	Eteinte	Clignotante 1 seconde

- Nota :** ✓ Vous disposez d'un bit système à l'adresse **%S6** qui permet d'obtenir une base de temps de **T#1s**. A l'aide de l'éditeur de données, lui associer la déclaration suivante :
sxBaseTemps1s at **%S6 : BOOL ;**
- ✓ L'éclairage simultané des composantes rouge et verte sur une colonne **RVB** permet d'obtenir une couleur jaune (ici plus orangée).

- 4.12 Télécharger de nouveau votre projet dans l'**API** et procéder à la recette de cette animation.
- 4.13 Dans la zone mémoire des données produites (**T->O**) par le contrôleur **PF200**, les octets (4, 5, 6 et 7) fournissent la durée d'immobilisation des palettes pleines avec une orientation de type **Little Endian** (**T#4s**), alors que les octets (8, 9, 10 et 11) la donnent avec une orientation de type **Big Endian**.

Reprendre votre traitement utilisateur afin d'utiliser ces données produites. On traitera d'abord le cas **Little Endian**. Pour retrouver l'interprétation de ces octets sous la forme d'un type **TIME**, vous serez amené à utiliser les techniques de **chevauchement** et d'**alias** vues en cours puisque le type **UNION** n'existe malheureusement pas sous **Control Expert**.

Annexe :

EcoStruxure Control Expert du constructeur *Schneider SA (Automate Modicon M340, M580, MC80, ...)*

La chaîne de développement *EcoStruxure Control Expert* offre la possibilité **de structurer un automatisme** (programme utilisateur) via des tâches (**Mast, Fast**, quatre tâches auxiliaires périodiques et des tâches événementielles). Chaque tâche est constituée d'une ou plusieurs **sections de programme** (conditionnelles ou non) ou **POUs (Program Organisation Unit)** écrits dans des langages de programmation normalisés **CEI-61131 (IL, LD, ST, SFC, FBD)**. Les **POUs** diffèrent **des sections** dans la mesure où ils peuvent avoir un contexte qui leur est propre (VAR INPUT, VAR IN_OUT, VAR OUTPUT, VAR (public /privé)).

Sous *Control Expert*, comme une **section** (ou un **POU**) ne peut pas appeler une autre **section** (un autre **POU**), c'est avec les **propriétés d'exécution conditionnelle des sections** (ou des **POUs**) qu'il est possible d'implanter une **commande hiérarchisée** à l'aide d'un **gmma**. L'application se décompose alors :

- A chaque **mode**, on associe une (ou plusieurs) **section(s) de programme conditionnelle(s)** (ou **POUs**) dont **l'évolution** est conditionnée par un **bit d'activité**.
- Le **gmma** est implanté dans une **section de programme** toujours **active** et **première à être scrutée** dans le cycle **API**. Il est décrit sous la forme d'une **machine d'états** en langage **SFC**. La **section postérieure** au **gmma** permet en fonction de l'étape **active** du graphe **SFC** de sélectionner la ou les **sections (POUs) conditionnelles** associées au **mode actif**. Le **gmma** joue ici le rôle de chef d'orchestre. La commande est donc hiérarchisée

A. Informations sur les sections en langage SFC

Pour chaque section **SFC**, un **objet** du type **opaque SFCCHART_STATE** est créé avec pour **nom** celui de la section. Cet **objet** permet de contrôler le graphe **SFC** via des **services spécifiques** tels que :

✓ BOOL InitChart (SFCCHART_STATE SfcToInit, BOOL bInit);	(* pour rendre actif l'état initial du SFC SfcToInit *)
✓ BOOL ClearChart (SFCCHART_STATE SfcToClear, BOOL bInit);	(* pour rendre inactives toutes les étapes du SFC SfcToClear *)
✓ voir aussi BOOL SfcCtrl_Instance(...),...	

Pour chaque **étape** d'une section **SFC**, un **objet** du type **SFCSTEP_STATE** est créé systématiquement avec pour **nom** celui de l'étape. L'utilisateur peut associer à chaque étape un **objet** du type **SFCSTEP_TIMES** s'il désire **activer un contrôle temporel** sur sa durée d'activation minimum ou maximum. Ce **contrôle temporel** optionnel **ne doit pas être utilisé** pour effectuer **des actions temporisées** mais pour **faciliter la mise au point, la maintenance, le diagnostic** et la **détection de défauts** :

- ✓ soit sur les capteurs : **un événement est perçu trop tôt, trop tard voir jamais**.
- ✓ soit sur les actionneurs : **un événement n'est jamais perçu et conduit à un blocage** (=> sortie par **time-out**)

Des **variables action** ou des **sections action** peuvent aussi être associées à chaque étape en spécifiant via le **qualificatif normalisé** leur nature (**N** : non mémorisée, **P1** : à l'activation, **P0** : à la désactivation, ...). Ces **variables action** ou **sections action** sont utiles pour la manipulation **d'objets internes** participant à la représentation de **l'état** (cf. paramétrage du modèle), mais deviennent vite lourdes d'écriture pour prendre en compte les **sécurités secondaires** sur les actions modifiant les sorties de l'**API**.

C'est pourquoi, il est préférable d'utiliser une section en langage **ST** (ou **LD**), **postérieure** au graphe **SFC** pour établir la synthèse des sorties puisque ce principe permet sans grande difficulté la mise en place indispensable des **sécurités secondaires**. Le **codage des actions** s'effectue alors comme dans une machine de **Mealy** en utilisant l'état via **les bits d'activité des étapes** (champ **x** de la structure **SFCSTEP_STATE**), et des entrées **capteur** pour les **sécurités**.

B. Détails des structures des objets du langage SFC

- ✓ La structure de données **SFCSTEP_TIMES** est un **enregistrement** (struct) constitué de **3 champs** accessibles en **lecture** et en **écriture** :

Type	Enregistrement :	SFCSTEP_TIMES
TIME delay		Temps de retard : durée séparant deux réactivations
TIME min		Temps de contrôle minimum : Il indique la durée minimale pendant laquelle l'étape doit au moins être active.
TIME max		Temps de contrôle maximum : Il indique la durée maximale pendant laquelle l'étape doit rester active.

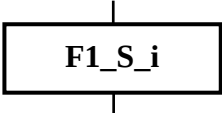
Nota : on doit toujours vérifier : **Temps de retard** < **Temps minimum de contrôle** < **Temps maximum de contrôle**

- ✓ La structure de données **SFCSTEP_STATE** est un enregistrement de **4 champs** accessibles en **lecture seule** :

Type	Enregistrement :	SFCSTEP_STATE
TIME t		temps d'activation de l'étape. Lorsque l'étape est désactivée, la valeur de cet élément est maintenue jusqu'à la prochaine activation de l'étape.
BOOL x		bit d'activité de l'étape (TRUE ⇔ Etape active, FALSE ⇔ Etape inactive)
BOOL tminErr		TRUE ⇔ dépassement par valeur inférieure du temps minimum de contrôle, FALSE sinon.
BOOL tmaxErr		TRUE ⇔ dépassement du temps de contrôle maximum, FALSE sinon.

- ✓ Pour établir **l'expression des conditions de transition**, celle-ci peut se faire **directement** si la condition porte sur **un seul objet**. Sinon, il faut établir une **section de transition**, l'ouvrir pour l'éditer dans le langage choisi.

On adoptera les règles suivantes pour établir les **noms** des **étapes** et des **sections de transition** :

✓ Pour désigner une étape : 	XX_S_i avec : XX : le nom du mode de fonctionnement qui y fait référence (exemple : Mode Production Normale : XX = F1). S : Step (étape) i : numéro de l'étape dans le mode.
✓ Préfixe d'une section de transition apparaissant dans le graphe SFC associé à un mode de fonctionnement :	XX_T avec : XX : le nom du mode de fonctionnement (exemple : F1_T_MiseEnService)